Proposta de Biblioteca Modular em C - Grafos

MAC0216 - Técnicas de Programação I | Trabalho Prático 2

A) Domínio escolhido

Grafo.

B) Objetivo da biblioteca

Criação, destruição, representação, manipulação e análise de grafos.

C) Quatro módulos previstos

Core - Criar e gerenciar.

Edges - Manipular arestas.

Traversal - Percorrer e verificar alcançabilidade.

Property - Analisar propriedades.

D) Funções públicas previstas

Core

```
Graph *graph_create(size_t n); // cria um grafo com n vértices
void graph_destroy(Graph *g); // libera a memória do grafo
size_t graph_get_vertices(const Graph *g); // retorna o número de vértices
Edge *graph_get_edges(const Graph *g, size_t *count); // retorna as arestas e sua quantidade
```

Edges

```
int graph_add_edge(Graph *g, int u, int v, int w); // adiciona a aresta u-v com peso w
int graph_remove_edge(Graph *g, int u, int v); // remove a aresta entre u e v
bool graph_has_edge(const Graph *g, int u, int v); // verifica se existe a aresta u-v
int graph_degree(const Graph *g, int u); // retorna o grau do vértice u
```

Traversal

```
bool graph_is_reachable(const Graph *g, int u, int v); // verifica se v é alcançável a partir de u
int graph_distance(const Graph *g, int u, int v); // retorna a menor distância de u até v
int *graph_all_distances(const Graph *g, int u); // retorna dist[v] para todos os vértices
size_t graph_component_size(const Graph *g, int u); // retorna o tamanho do componente de u
```

Properties

```
bool graph_is_connected(const Graph *g); // verifica se o grafo é conexo
bool graph_has_cycle(const Graph *g); // verifica se o grafo possui ciclo
bool graph_is_tree(const Graph *g); // verifica se o grafo é uma árvore
bool graph_is_forest(const Graph *g); // verifica se o grafo é uma floresta
```

E) Exemplo de utilização pela aplicação cliente

```c
#include <stdio.h>
#include "graph.h"

int main(void) {
    Graph *g = graph_create(6);
    if (g == NULL)
        return 1;

    graph_add_edge(g, 0, 1, 3);
    graph_add_edge(g, 2, 4, 2);
    graph_add_edge(g, 2, 0, 1);

    printf("0 chega a 2? %d\n", graph_is_reachable(g, 0, 2));
    // Saída esperada: 1

    printf("Distancia de 2 a 1: %d\n", graph_distance(g, 2, 1));
    // Saída esperada: 4

    graph_destroy(g);
    return 0;
}
```

F) Convenções adotadas

- Grafo não direcionado com vértices de 0 até N-1.
- Implementar struct Graph.
- Implementar struct Edge.
- Implementar funções públicas em vez de métodos de struct Graph.
- Arestas ponderadas.